A Big Data Framework for Price Tracking and Product Matching in Sri Lankan E-Commerce

P.K.K. Dias

June 23, 2026

Contents

Chapter 1

Introduction

Chapter 2

Literature Review

2.1 Anatomy and structure of websites and web pages

2.1.1 Structure of a website

The structure of a web page generally follows one of four structural models [1].

1. **Hierarchical (tree structure)**: This is the most common model, mirroring human cognitive processes and the way we organize information. It consists of a root node (the homepage) with branches leading to category pages, which further branch out to product pages and other content. This structure is intuitive for users and allows for easy navigation.
2. **Sequential (linear structure)**: In this model, pages are arranged in a linear sequence, often used for storytelling or step-by-step guides. Users navigate through the content in a predetermined order, which can be effective for certain types of content but may limit user freedom. This style is common in e-commerce checkout processes, where users are guided through a series of steps to complete a purchase.
3. **Matrix (grid structure)**: This model allows for multiple pathways between pages, with links connecting various nodes in a non-linear fashion. It is often used in content-rich websites, such as news sites or social

media platforms, where users can navigate through different topics and categories without a strict hierarchy.

4. **Database (dynamic structure)**: In this model, content is generated dynamically from a database based on user queries or interactions. It is common in e-commerce sites, where product pages are generated based on user searches or filters, and in social media platforms, where user-generated content is displayed based on relevance or popularity.

2.1.2 Anatomy of a web page

A web page is collection of elements that are common components of the overall website (site-level components), and elements that are specific to the individual page (page-level components) [2].

Site-level components

Site-level components are elements that are typically present across multiple pages of a website, providing consistency and aiding in navigation. These include:

- **Header**: The header is located at the top of the page and often contains the website's logo, navigation menu, search bar, and sometimes contact information or social media links. It serves as a consistent element across the site, allowing users to easily navigate to different sections of the website.
- **Footer**: The footer is located at the bottom of the page and often contains additional navigation links, contact information, copyright notices, and sometimes social media links. Like the header, it provides consistency across the site and can help users find important information or navigate to other sections.

Page-level components

Page-level components are elements that are specific to the individual page and can vary widely depending on the type of page and its purpose. These include:

- **Main content area**: This is the central part of the page where the primary content is displayed. It can include text, images, videos, and other media, and is the main focus of the page.
- **Sidebar**: A sidebar is an additional column that can be located on either side of the main content area. It often contains supplementary information, such as related articles, advertisements, or additional navigation links.
- **Hero section**: A hero section is a prominent area at the top of the page, often featuring a large image or video, along with a headline and call-to-action button. It is designed to capture the user's attention and convey the main message or purpose of the page.

2.1.3 HTML tags and the document object model (DOM)

HTML (HyperText Markup Language) is the standard language used to create and structure web pages. It uses a system of tags to define the structure and content of a web page, allowing browsers to render the page correctly. Semantic HTML involves the use of HTML tags such as `<header>`, `<nav>`, `<main>`, `<article>`, and `<footer>` to provide meaning to the content and structure of the page, which can improve accessibility and SEO (Search Engine Optimization).

The Document Object Model (DOM) is a programming interface for HTML and XML documents. It represents the structure of a web page as a tree of nodes, where each node corresponds to an element in the HTML document. The DOM allows developers to manipulate the content and structure of a web page dynamically using JavaScript, enabling interactive features and dynamic content updates without requiring a page reload [3].

2.1.4 Information hierarchy in e-commerce websites

E-commerce sites are usually structured around a conversion funnel designed to move visitors from awareness to purchase [5].

- **Homepage**: The homepage serves as the entry point to the website and provides an overview of the site's offerings, often featuring promotions,

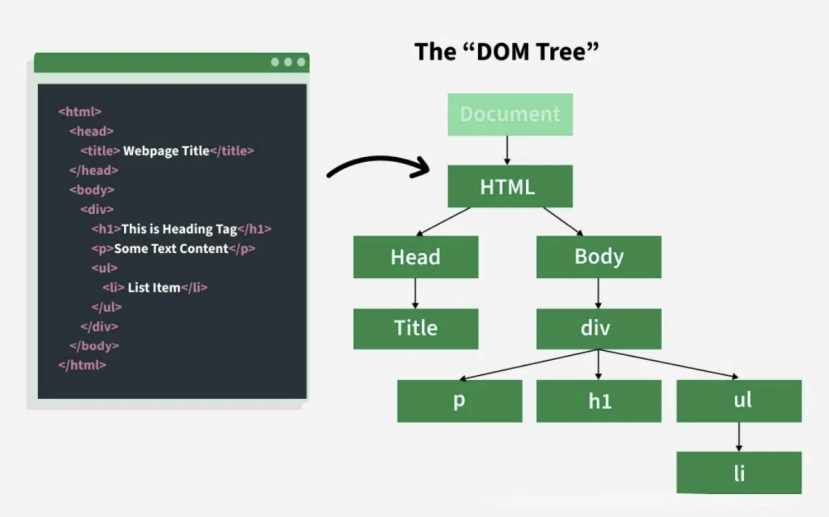

Figure 2.1: HTML and the corresponding DOM tree [4]

popular products, and links to key categories. It is designed to capture the user's attention and encourage them to explore further.

- **Product Listing Page (PLP)**: The PLP displays a list of products within a specific category or based on a search query. It typically includes filters and sorting options to help users find the products they are interested in. The PLP is designed to facilitate product discovery and encourage users to click through to individual product pages.
- **Product Detail Page (PDP)**: The PDP provides detailed information about a specific product, including images, descriptions, specifications, pricing, and customer reviews. It is designed to provide all the information a user needs to make a purchasing decision and often includes a call-to-action button to add the product to the cart or proceed to checkout.

2.1.5 The semantic web and structured data

The semantic web is an extension of the current web that aims to make data on the web more machine-readable and understandable by providing a standardized way to represent and link data using technologies such as RDF (Resource Description Framework) and OWL (Web Ontology Language). Structured data is a way of annotating web content with specific tags and

formats that provide additional context and meaning to the content, making it easier for search engines and other applications to understand and process the information on the web page. One common format for structured data is JSON-LD (JavaScript Object Notation for Linked Data), which allows webmasters to embed structured data in a web page using a JSON format that is easy for both humans and machines to read and understand [6].

In e-commerce websites, structured data can be used to provide detailed information about products, such as price, availability, reviews, and other attributes, which can enhance the visibility of the products in search engine results and improve the user experience by providing rich snippets in search results.

```
{
  "@context": {
    "foaf": "http://xmlns.com/foaf/0.1/",
    "title": "foaf:title",
    "name": "foaf:name",
    "homepage": {
      "@id": "foaf:workplaceHomepage",
      "@type": "@id"
    }
  },
  "@id": "http://me.markus-lanthaler.com",
  "@type": "foaf:Person",
  "title": [
    {"@value":"Dipl.Ing.", "@language":"de"},
    {"@value":"MSc", "@language": "en"}
  ],
  "name": "Markus Lanthaler",
  "homepage": "http://www.tugraz.at/"
}
```

Figure 2.2: Example of a JSON-LD document [6]

Web crawling is the process of systematically browsing the web and extracting content from its pages. It is a fundamental technique used in various applications, most notably in search engines, where it is used to index the web and make it searchable. Another common use of web crawling is in data mining and web scraping, where it is used to collect data from websites for analysis or other purposes. More recently, web crawling has also been used in the context of training large language models, where it is used to gather vast amounts of text data from the web to train these models. Web crawlers, also known as spiders or bots, are automated programs that perform this task.

Crawlers typically traverse the web by following hyperlinks from one page to another, starting from a set of seed URLs [7]. They can be designed to crawl the entire web or to focus on specific domains, topics, or types of content.

A standard crawler operates in several stages [8]:

1. **Seed URLs**: The crawler starts with a list of initial URLs, known as seed URLs, which are the starting points for the crawling process.
2. **Downloading**: The crawler sends HTTP requests to the URLs and downloads the content of the web pages. The downloaded content is stored in local storage or cloud storage for further processing.
3. **Extracting links**: The crawler parses the downloaded pages to extract hyperlinks, which are then added to a crawl frontier.

A crawl frontier is a data structure that manages the list of URLs to be crawled, ensuring that the crawler does not revisit the same URL multiple times and that it adheres to any specified crawling policies (e.g., depth limits, domain restrictions). The crawl frontier can also be used as a queue to manage iterative and repeated crawling, allowing the crawler to revisit pages at regular intervals to check for updates, maintaining the freshness of the dataset [7].

Traditional web crawlers worked by sending HTTP requests to web servers and downloading the HTML content of web pages. However, such crawlers may struggle with modern websites that heavily rely on JavaScript and AJAX to render content dynamically via client-side rendering [9]. Modern web crawlers often need to use headless browsers or tools like Selenium, Puppeteer, or Playwright to handle JavaScript content and ensure that they can access the full content of web pages [10]. These tools allow the crawler to execute JavaScript and render the page as a regular browser would, while consuming fewer resources than a full browser, enabling the crawler to access dynamic content effectively.

It is also important to consider the legal and ethical implications of web crawling.

1. **Webmaster intent**: Webmasters may not want their content to be crawled or indexed, and they can use the `robots.txt` file to specify which parts of their site should not be accessed by crawlers [11].

2. **Server load**: Crawling can put a significant load on web servers, especially if the crawler is not designed to be respectful of the site's resources. It is important for crawlers to implement politeness policies, such as rate limiting, to avoid overwhelming servers [7].

3. **Data privacy**: Crawling and scraping can raise privacy concerns, especially if personal data is collected without consent. It is crucial to ensure that any data collected through crawling is handled in compliance with relevant data protection laws and regulations, such as the General Data Protection Regulation (GDPR) in the European Union [12].

A significant challenge in modern web crawling is the use of anti-bot measures by websites, such as CAPTCHAs, IP blocking, and rate limiting, which can hinder the crawling process [13]. Advanced security services such as Cloudflare employ fingerprinting techniques to identify and block automated traffic, making it increasingly difficult for crawlers to access content on certain websites [14]. Crawlers need to be designed to navigate these challenges while still adhering to ethical and legal guidelines Moreover, content hidden behind paywalls or login screens can also pose challenges for crawlers, as they may require authentication or subscription to access the content. Most crawlers, including Google's [15] generally do not access such content, unless explicit permission and credentials are provided by the site owner.

2.1.6 Summary

Web crawling involves a range of technical and ethical considerations. Static crawlers are simple and efficient but fail on JavaScript-heavy pages, while headless browser-based crawlers handle dynamic content and client-side rendering at highest resource costs. Anti-bot measures and legal/ethical constraints add further complexity that must be carefully managed in any crawling system. Table 2.1 summarises the key crawler types and their trade-offs.

2.2 Boilerplate detection and removal

A web page is a document that contains both content and presentation information. The content of a web page is the information that is relevant to the user, while the presentation information is the information that is used

Crawler Type	Strengths	Limitations
Static HTTP crawler	Lightweight, fast, low resource usage	Cannot handle JavaScript / client-side rendering
Headless browser	Handles dynamic content; renders JS fully	High resource usage; slower at scale

Table 2.1: Summary of web crawler types and their trade-offs

to display the web page. Boilerplate refers to the presentation information that is not relevant to the user, such as navigation menus, advertisements, and other non-content elements.

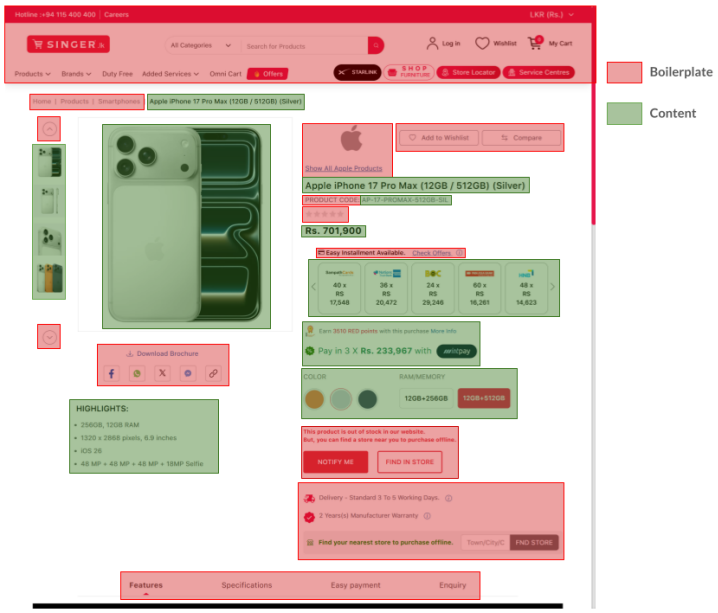

Figure 2.3: Boilerplate vs. content on a web page

Boilerplate detection and removal is the process of identifying and removing boilerplate from a web page so that only the relevant content remains. This is an important task for many applications, such as web scraping, information retrieval, and natural language processing as boilerplate/template content can interfere with the accuracy of these applications.

There are primarily two approaches to boilerplate detection and removal [16]:

1. **Site-Level Methods**: These methods analyze multiple pages from the same website to identify common patterns and structures that are likely to be boilerplate. By comparing the structure and content of different pages, site-level methods can effectively identify and remove boilerplate elements that are consistent across the site.

2. **Page-Level Methods**: These methods analyze each page individually to identify boilerplate elements based on features such as link density, text density, and the structure of the HTML document.

2.2.1 Site-level methods

Site Style Trees (SST)

Using the DOM from a sample of pages from a website, a site style tree (SST) can be constructed to represent the common structure of that specific website. For each unique subtree which appears in the SST, a count is maintained to track how many pages contain that subtree. Subtrees that appear in a large number of pages are likely to be boilerplate, while those that appear in fewer pages are more likely to be content. By analyzing the SST, it is possible to identify and remove boilerplate elements from the pages of the website, leaving only the relevant content. [17]

Using subtree hashes

Similar to site style trees, subtree hashes can be used to identify boilerplate elements across multiple pages of a website. By computing a hash for each subtree in the DOM of a page, it is possible to compare these hashes across different pages to identify common subtrees that are likely to be boilerplate. Subtrees that have the same hash across multiple pages are likely to be boilerplate, while those that have unique hashes are more likely to be content. This method can be computationally efficient and effective in identifying boilerplate elements. [18]

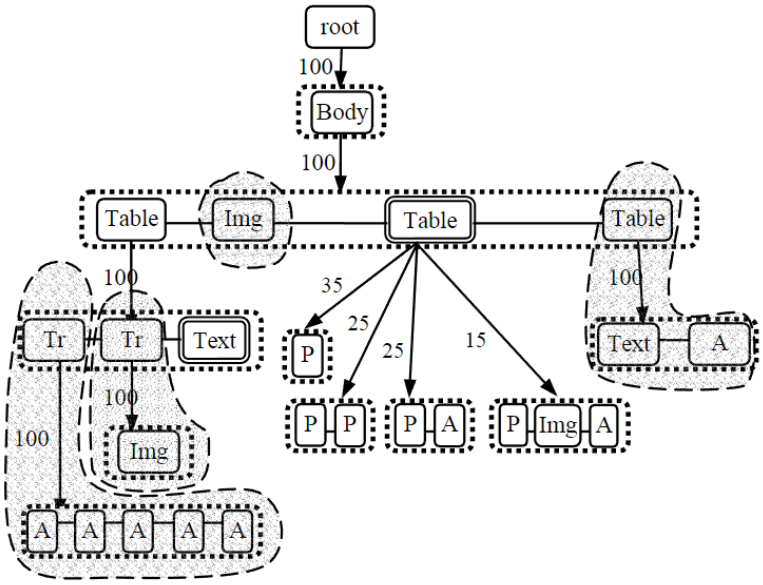

Figure 2.4: An example of site style trees (SST) [17]

2.2.2 Page-level methods

Body Text Extraction (BTE)

This is a heuristic-based method based on the observation that the main content of a web page usually contains long streams of uninterrupted text with a low HTML tag density. By analyzing the structure of the HTML document and identifying areas with high text density and low tag density, it is possible to extract the main content of the page. While this is a simple method, one major drawback is that modern web pages are extremely complex and may not always follow this pattern, leading to inaccurate results. [19]

Shallow text features

This method first chunks the web page into smaller sections based on the structure of the HTML document. It then computes various features for each chunk, such as link density (the ratio of links to text), text density (the ratio of text to HTML tags), and other structural features. High link density is generally indicative of boilerplate, while high text density is more indicative of content. [20]

This also builds on the concept of functional short text vs descriptive long text, where functional short text, characterized by short, incomplete sentences (e.g., home, contact us) is more likely to be boilerplate, while descriptive long text, characterized by full sentences and higher complexity is more likely to be content. [20]

Supervised machine learning techniques

While BTE and shallow text features can be effective in some cases, they may not always be sufficient to accurately identify boilerplate elements, especially on modern web pages with complex structures. Supervised machine learning techniques, such as neural networks and multi-layer perceptrons (MLPs), can be trained on labeled datasets of web pages to learn more complex patterns and features that distinguish boilerplate from content. By using a large and diverse training dataset, these models can learn to generalize well to new web pages and can be more effective in accurately identifying boilerplate elements. These techniques generally show a significant improvement in performance compared to heuristic-based methods, especially on modern web pages with complex structures. [21]

Two examples of supervised machine learning techniques for boilerplate detection and removal are Web2Text [19], which uses Convolutional Neural Networks (CNNs) to predict probabilities for chunks of a page, and the transition between blocks, and BoilerNet [22], which use bidirectional LSTMs that learn dependency between blocks and their neighbours.

2.2.3 Performance improvements

Removing boilerplate content from web pages have shown to significant performance gains in downstream tasks such as web page classification and clustering, as it allows these tasks to focus on the relevant content of the page rather than being distracted by irrelevant boilerplate elements. In addition, it also reduces the amount of data that needs to be processed, which can lead to faster processing times and improved efficiency in these tasks.

Using site style trees (SSTs) to remove boilerplate has been shown to improve the accuracy in web page classification tasks from 0.625 to 0.954 [17]. Using the same method, the average F1-score for k-means clustering also improved from 0.506 to 0.751.

2.2.4 Summary

Boilerplate detection and removal is a problem with a number of possible approaches ranging from simple rule-based models to deep learning models. Site-level methods are effective when multiple pages from the same website are available, while page-level methods operate on individual pages without any prior site knowledge. Supervised machine learning techniques offer the highest accuracy but require labeled training data. Table 2.2 summarises the key approaches reviewed in this section.

Method	**Type**	**Strengths**	**Limitations**
Site Style Trees (SST) [17]	Site-level	Cross-page detection, improves classification & clustering	Needs multiple pages from same site
Subtree hashes [18]	Site-level	Efficient, easy to implement	Needs multiple pages, fragile to DOM changes
Body Text Extraction (BTE) [19]	Page-level	No training data, lightweight	Poor accuracy on complex pages
Shallow text features [20]	Page-level	Interpretable, no training data	Hand-crafted heuristics, limited generalisation
Web2Text [19]	Page-level	Learns complex patterns	Requires labeled data
BoilerNet [22]	Page-level	Models block dependencies, best accuracy	Requires labeled data, higher complexity

Table 2.2: Summary of boilerplate detection and removal approaches

2.3 Web page classification

Web page classification is the process of categorizing web pages in one or more predefined classes based on their content, structure, and visual properties.

This can be considered to be a pre-processing task of information extraction/retrieval, as the method and types of information extraction/retrieval can be different for different types of web pages. For example, the method of information extraction for a e-commerce website's product page may be different from the method of information extraction for it's contact details page. Web page classification, while fundamentally can be considered to be a text document classification task, is a unique task due to the complex structure of web pages and the presence of both content and presentation information.

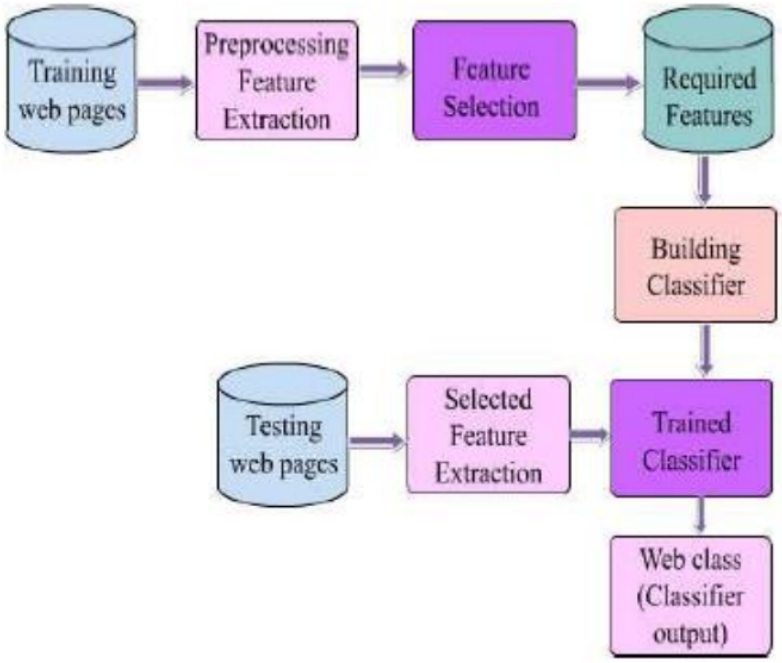

Figure 2.5: General process for web page classification [23]

2.3.1 Web page classification techniques

Web page classification techniques can be broadly categorized into three categories:

1. Keyword-based techniques: These techniques rely on the presence of specific keywords or phrases in the web page and its frequency to classify it into a particular category.

2. Supervised learning techniques: These techniques use labeled training data to train a machine learning model to classify web pages into different categories based on their content and structure.

3. LLM based techniques: These techniques utilize large language models (LLMs) to classify web pages.

Keyword-based techniques

Keyword-based techniques are fairly simple, rule based approaches which do not require a labeled training or training. Documents are assigned to categories based on the highest match count of specific keywords, or on keyword density (the ratio of the number of occurrences of a keyword to the total number of words in the document). While such techniques are simple and easy to implement, they can be inaccurate and perform poorly on more complex pages [24]. More advanced techniques such as Term Frequency - Inverse Document Frequency (TF-IDF) can be used to improve the performance of keyword-based techniques by giving more weight to keywords that are more relevant to a particular category and less weight to common keywords that appear in many categories. Combining TF-IDF with traditional machine learning classifiers have shows to have strong performance in web page classification tasks [25].

Supervised learning techniques

Supervised learning techniques for web page classification involve training a machine learning model on a labeled dataset of web pages, where each web page is associated with a specific category or class. The model learns to classify web pages based on the features extracted from the content and structure of the web pages. Classical algorithms such as Naive Bayes, Support Vector Machines (SVM), k-Nearest Neighbour (kNN) and Decision Trees have been used for web page classification, while SVMs have been shown to be particularly effective for this task [26].

More recently, deep learning techniques utilizing Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs) have been applied to web page classification tasks, showing improved performance over traditional machine learning algorithms. Transformer-based models such as BERT (Bidirectional Encoder Representations from Transformers) have also been applied to web page classification tasks, showing strong performance due to

their ability to capture contextual information in the text. However, more simple models usually offer a better efficiency-performance trade-off, and are often preferred in settings where computational resources may be limited [27].

LLM based techniques

Large language models (LLMs) have shown to be effective in various natural language processing tasks, including web page classification. While this is a very complex technique, the existence of foundational models by OpenAI, Google, Anthropic and others, has made it more accessible and easier to implement via an API call. A significant benefit of using LLMs, is that it can be used for zero-shot classification, which means it can be used when there are no training example available for the specific classification task. Another advantage is that it can handle very long documents (contexts) with high accuracy. A major drawback though, is the high cost of this method as API calls to LLMs can be expensive, especially for long, feature rich documents such as web pages. Moreover, the per-page inference time can also be significantly higher compared to traditional machine learning techniques, which can be a concern in settings where real-time classification is required or when processing large volumes of web pages [24]

2.3.2 Techniques without negative examples

As web page labelling is generally a manual and time-consuming process, in some cases, it may be difficult to have a complete labeled dataset consisting of both positive and negative classes (e.g. product pages vs. non-product pages). It can be easier to just have a set of positive examples (e.g. product pages) without having a complete set of negative examples (e.g. non-product pages). In such cases, the Positive Example Based Learning (PEBL) framework [28] can be to identify strong negatives from the unlabeled data and then train a classifier using the positive examples and the identified strong negatives. This approach can be effective in situations where it is difficult to obtain a complete labeled dataset.

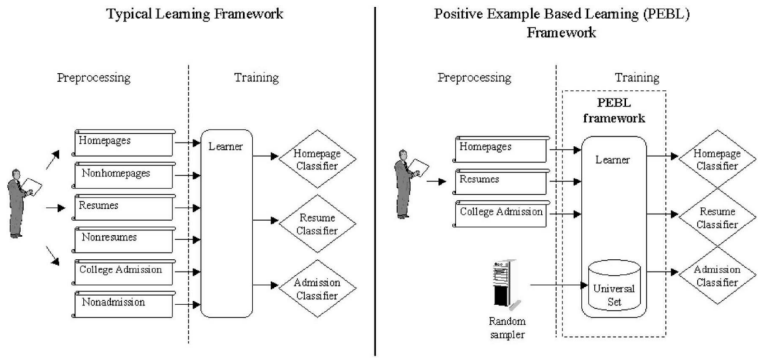

Figure 2.6: Typical framework vs. using PEBL [28]

2.3.3 Effect of noise removal

In the context of web page classification, noise refers to the irrelevant information on a web page that can interfere with the accuracy of the classification process. This can include boilerplate content, advertisements, navigation menus, and other non-content elements. The presence of noise can lead to misclassification of web pages, as the classifier may be influenced by the irrelevant information rather than the relevant content. By removing noise from web pages, the classifier can focus on the relevant content, which can lead to more accurate classification results. In [29], it was indicated that performance of web page classification improved by 10% to 15% after noise removal.

2.3.4 MarkupLM

MarkupLM [30] is a pre-trained transformer based model designed for markup-language based documents such as HTML and XML. Unlike standard techniques, which treat web pages as a block of text, MarkupLM takes into account the structure of the web page by incorporating the HTML tags and their relationships into the model. It introduces XPath embeddings, which capture for each node, it's position in the DOM tree, its relationship with other nodes, and the HTML tag information. This also allows MarkupLM to capture the hierarchical structure of web pages, which can be important for understanding the context and meaning of the content on the page. This has resulted in significant performance improvements compared to text-only based techniques and models.

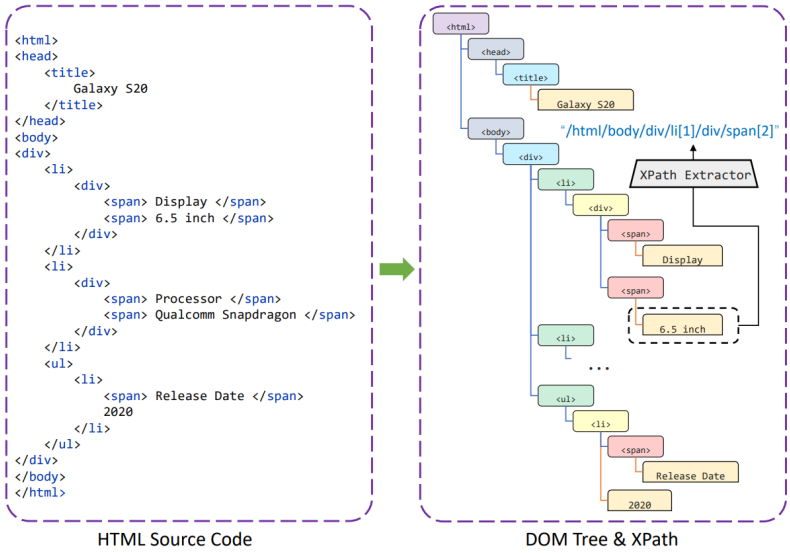

Figure 2.7: DOM tree and XPath in HTML documents [30]

2.3.5 E-commerce web page classification

E-commerce web page classification is a specific application of web page classification that focuses on categorizing web pages related to e-commerce, such as product pages, product list pages, blog pages etc. This can be particularly challenging as e-commerce sites contain complex and repetitive structures, which can also vary significantly across different sites. In addition, the presence of noise such as advertisements, navigation menus, and other non-content elements can also hinder the classifier's performance. In [26], it was shown that Multi-Layer Perceptron (MLP) based models and SVMs can achieve a strong performance on e-commerce web page classification tasks.

2.3.6 Summary

Web page classification techniques vary in their complexity, data requirements, and resource requirements. Keyword-based methods are simple but inaccurate on complex pages, while supervised learning and transformer-based models offer stronger performance while requiring extensive labelled training data and compute resources. LLMs provide a zero-shot alternative but are expensive at scale. Table 2.3 summarises the key approaches reviewed

in this section.

Technique	Strengths	Limitations
Keyword-based (TF-IDF)	Simple, no training data, fast	Low accuracy on complex pages
Classical ML (SVM, kNN, Naive Bayes)	Well-studied, strong baseline, efficient	Requires labeled data, limited context understanding
Deep learning (CNN, RNN)	Captures sequential patterns, better generalisation	High data and compute requirements
Transformer-based (BERT, MarkupLM)	Captures context, leverages HTML structure	Computationally expensive, needs fine-tuning
LLM-based	Zero-shot, handles long context	High API cost, slow inference at scale
PEBL (positive examples only)	No negative examples needed	Less accurate than fully supervised methods

Table 2.3: Summary of web page classification techniques

2.4 Web information extraction

Web information extraction is the process of extracting structured data from unstructured web content. While traditionally, this was done using rule-based approaches, where developers would write specific rules to identify HTML elements via their XPath or CSS selectors, modern approaches have shifted towards machine learning and deep learning techniques. These methods leverage the underlying structure of the webpage, such as the Document Object Model (DOM), to identify and extract relevant information. For instance, models can be trained to recognize patterns in the DOM tree, allowing them to extract data even when the webpage layout changes, which is a common occurrence on the web. Another requirement is for zero-shot extraction, where the model can extract information from webpages it has never seen before, without needing to be retrained for each new website. In the context of e-commerce, web information extraction involves tasks such as extracting product details (e.g., name, price, description) from product

pages, which can be challenging due to the diversity of webpage designs and the presence of boilerplate content.

2.4.1 Embedded semantic metadata

More modern websites, especially those build on top of popular frameworks and content management systems, often include embedded semantic metadata in their HTML code to provide additional context about the content of the page. This metadata is used to provide search engine crawlers and other applications with structured information about the content of the page, which can be used to improve search results and enable rich snippets in search engine results pages (SERPs).

Microdata, RDFa, and JSON-LD are common formats for embedding semantic metadata in HTML.

- **Microdata** [31] is a specification for embedding structured data in HTML documents using a simple syntax. It allows developers to annotate their HTML with additional attributes that provide information about the content of the page.
- **RDFa** [32] (Resource Description Framework in Attributes) is a W3C recommendation that provides a way to embed RDF (Resource Description Framework) data in HTML documents using attributes. It allows for more complex and flexible annotations compared to Microdata.
- **JSON-LD** [33] (JavaScript Object Notation for Linked Data) is a lightweight format for encoding linked data using JSON. It is often used to embed structured data in web pages, and it can be easily parsed by search engines and other applications.

Most of these metadata formats are based on the Schema.org [34] vocabulary, which provides a standardized set of types and properties for describing various entities and their relationships on the web. For example, a product page on an e-commerce website might include JSON-LD markup that describes the product's name, price, description, and availability, as shown in Listing 2.1.

```
1 {
2   "@context": "https://schema.org/",
```

```
  "@type": "Product",
  "@id": "https://lifemobile.lk/product/honor-magic-v5-5g-16
     gb-ram-512gb/#product",
  "name": "Honor Magic V5 5G 16GB RAM 512GB",
  "url": "https://lifemobile.lk/product/honor-magic-v5-5g-16
     gb-ram-512gb/",
  "description": "7.95 inches Display, 5820mAh silicon-carbon
      battery, Snapdragon 8 Elite, 1 Year Company Warranty",
  "image": "https://lifemobile.lk/wp-content/uploads/2025/11/
     Honor-Magic-V5-5G-16GB-RAM-512GB.jpg",
  "sku": "honor-magic-v5-5g-16gb-ram-512gb",
  "offers": [
    {
      "@type": "Offer",
      "priceSpecification": [
        {
          "@type": "UnitPriceSpecification",
          "price": "634285.71",
          "priceCurrency": "LKR",
          "valueAddedTaxIncluded": false,
          "validThrough": "2027-12-31"
        }
      ],
      "priceValidUntil": "2027-12-31",
      "availability": "http://schema.org/InStock",
      "url": "https://lifemobile.lk/product/honor-magic-v5-5g
         -16gb-ram-512gb/",
      "seller": {
        "@type": "Organization",
        "name": "Life Mobile",
        "url": "https://lifemobile.lk"
      }
    }
  ]
}
```

Listing 2.1: Example JSON-LD product markup from an e-commerce webpage with the Schema.org vocabulary

Of the three most common metadata formats, JSON-LD is the most widely used for embedding structured data in web pages, particularly for e-commerce websites. An analysis of the 100 most popular e-commerce websites in the US and UK in 2023 found that only 45% of URLs did not contain JSON-LD metadata, and 27% contained unstructured JSON-LD metadata with errors. [35]

Therefore, while embedded semantic metadata can be a valuable source of

structured data for web information extraction, it is not universally present or consistently implemented across all websites, which can limit its effectiveness as a sole method for extracting information from the web.

2.4.2 Vision-based approaches

While the Document Object Model (DOM) provides a structured representation of a webpage, it can be complex and noisy, and usually fails to reflect the actual semantic grouping of content as perceived by human users. Vision-based approaches to web information extraction address this issue by treating the webpage as an image and leveraging computer vision techniques to identify and extract relevant information. The **Vision-based Page Segmentation (VIPS) algorithm** [36] was developed to segment a webpage into visually coherent blocks, which can then be analyzed to extract structured data.

The VIPS algorithm works by building a hierarchical tree structure of the webpage based on its visual layout, rather than its DOM structure. Each node in the tree represents a visually coherent block of content, and the algorithm uses various visual cues such as font size, color, and spatial relationships to determine how to segment the page.

2.4.3 DOM tree node classification

Another approach to web information extraction is to treat the problem as a classification task, where each node in the DOM tree is classified as either relevant or irrelevant for the information extraction task at hand. This approach can be implemented using machine learning or deep learning models, which can learn to identify relevant nodes based on features such as the node's tag name, attributes, text content, and its position in the DOM tree. Graph Neural Networks (GNNs) have been used to model the relationships between nodes in the DOM tree, allowing for more effective classification of relevant nodes.

ZeroShotCeres [37] is a model that uses a GNN to classify nodes in the DOM tree for web information extraction. It is designed to perform zero-shot extraction, meaning it can classify nodes on webpages it has never seen before, without needing to be retrained for each new website. The model is

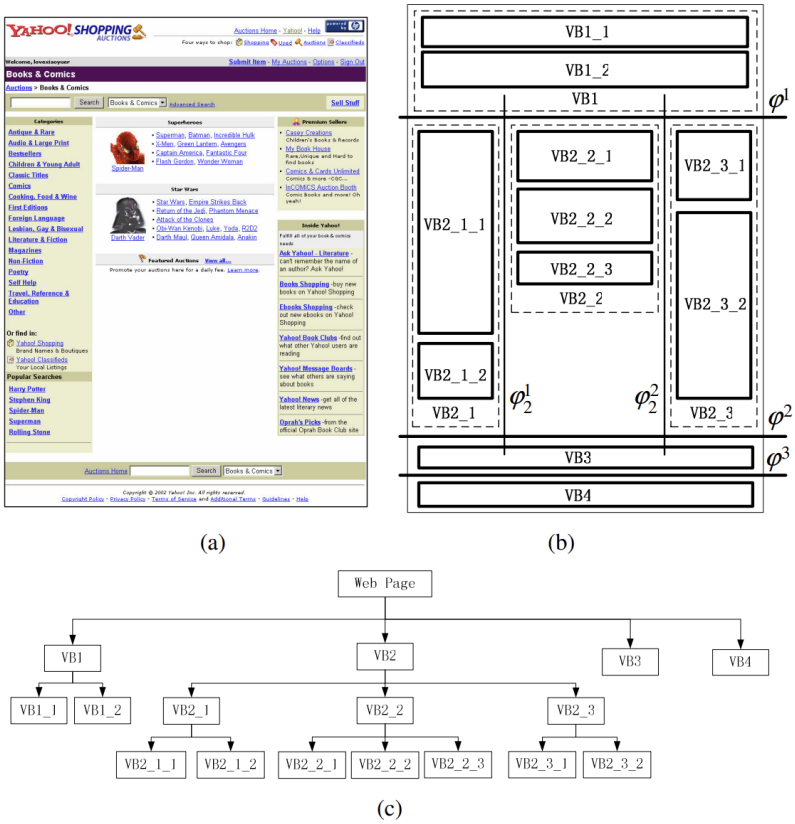

Figure 2.8: Image segmentation of a webpage using the VIPS algorithm [36]

trained on a large dataset of webpages with labeled nodes, and it learns to generalize from this training data to classify nodes on new webpages based on their features and their relationships to other nodes in the DOM tree.

AttriSage [38] uses a similar approach to extract product attributes from e-commerce webpages by classifying nodes by treating the task as a multi-label node classification problem, where each node can be classified with multiple labels corresponding to different product attributes (e.g., name, price, description). It uses a GNN to model the relationships between nodes in the DOM tree, allowing it to effectively classify nodes based on their features and their relationships to other nodes, even when the webpage layout varies significantly across different e-commerce websites.

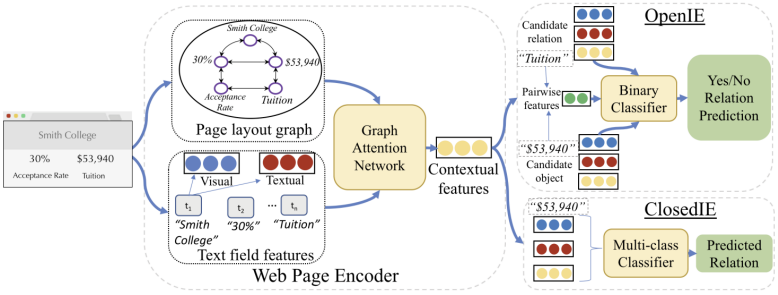

Figure 2.9: Node classification using ZeroShotCeres [37]

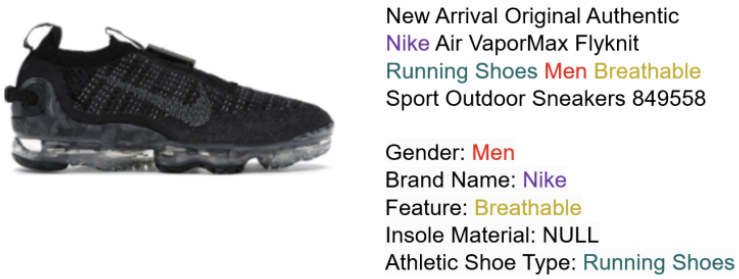

Figure 2.10: Product attribute extraction [38]

2.4.4 Markup and layout-based approaches

Recent transformer architectures have been developed that integrate the structural hierarchy of HTML directly into language representations, allowing for more effective web information extraction by leveraging both the textual content and the structural layout of the webpage. Unlike traditional models which only encode the textual content of a node, these models also encode the position of the node in the DOM tree, which can provide valuable context for understanding the relationships between different elements on the page.

MarkupLM [30] encodes both the textual content of a node and its position in the DOM tree using a specialized XPath embedding layer. It is a pretrained language model that can be fine-tuned for various web information extraction tasks. MarkupLM also treats information extraction as a token classification task, where each token in the input sequence is classified as

either relevant or irrelevant for the extraction task, allowing it to effectively identify and extract relevant information from the webpage. Since MarkupLM does not actually render the webpage, it is very lightweight and can be used for large-scale web information extraction without the overhead of rendering webpages, while still leveraging the structural information provided by the DOM tree.

MarkupLM uses BERT architecture and is pretrained on 24 million web pages from the Common Crawl [39] dataset, which allows it to learn a wide range of patterns and structures commonly found in web pages, making it effective for zero-shot extraction on new websites.

WebFormer [40] is a transformer architecture that integrates the structural hierarchy of HTML directly into language representations by encoding the position of each node in the DOM tree using a specialized embedding layer. There are three types of tokens used in WebFormer in the input layer.

- **Field tokens**represent the different fields that are being extracted from the webpage, such as product name, price, description, etc.
- **HTML tokens** represent the HTML path of each node in the DOM tree, which provides information about the structure and layout of the webpage, allowing the model to understand the relationships between different elements on the page.
- **Text tokens** represent is an embedding of the textual content of each node in the DOM tree, which provides information about the content of the webpage.

WebFormer uses four specialized attention patterns (HTML-to-HTML, HTML-to-Text, Text-to-HTML, and Text-to-Text) to allow the model to effectively leverage both the structural information provided by the HTML tokens and the content information provided by the text tokens, enabling it to perform accurate web information extraction even when the webpage layout varies significantly across different websites.

Benchmark	Metric	MarkupLM	WebFormer
SWDE (1 seed site)	F1	82.11%	–
SWDE (5 seed sites)	F1	97.37%	92.46%
SWDE	EM	–	86.58%
SWDE (Products)	F1	–	83.37%
SWDE (Products)	EM	–	80.67%
WebSRC	F1	74.47%	–
WebSRC	EM	68.39%	–

Table 2.4: Performance comparison of MarkupLM and WebFormer on standard benchmarks

Performance comparison

2.4.5 Multimodal approaches

Multimodal approaches to web information extraction leverage multiple sources of information, such as the textual content and the visual layout of the webpage, to improve the accuracy and robustness of information extraction.

LayoutLMv3 [41] is one such model that encodes text (via OCR) and layout coordinates and image patches. It is a pretrained multimodal transformer model that can be fine-tuned for various web information extraction tasks. LayoutLMv3 uses a combination of text, layout, and image information to perform information extraction, allowing it to effectively identify and extract relevant information from the webpage even when the layout varies significantly across different websites. While LayoutLMv3 can achieve high accuracy on web information extraction tasks, it is computationally expensive due to the need to render the webpage and perform OCR to extract text, which can limit its scalability for large-scale web information extraction tasks.

2.4.6 LLM-based approaches

Large Language Models (LLMs) excel at zero-shot information extraction tasks, where the model can extract information from webpages it has never

seen before without needing to be specifically trained. LLMs can be prompted with the HTML content of a webpage and a specific extraction task, and they can generate structured data based on the provided input. For example, a prompt might include the HTML content of a product page and a request to extract the product name, price, and description, and the LLM would generate a structured output containing this information.

A drawback of LLM-based approaches is that they can be computationally expensive, especially for large webpages with complex layouts, and they may not always produce accurate results, particularly when the webpage contains a lot of noise or irrelevant information. Therefore, it is essential that the number of tokens provided as input to the LLM is limited, which can be achieved through boilerplate removal techniques that filter out irrelevant content from the webpage before feeding it into the LLM, and by converting documents into Markdown format, which can help to preserve the structural information of the webpage while reducing the amount of token introduced due to encoding HTML tags [42].

2.4.7 Summary

Web information extraction approaches vary significantly in their reliance on structure, visual layout, and scale. Rule-based methods are precise but brittle, while machine learning and transformer-based models offer greater generalisability at the cost of increased complexity. Embedded semantic metadata provides a lightweight extraction path but is inconsistently adopted across the web.

2.5 Product matching

Product matching is the task of identifying and linking different web pages which represent the same underlying product across different e-commerce websites. These kinds of tasks are also commonly known as entity resolution, record linkage, or deduplication. Brute-forcing product matching is a computationally expensive task, with a quadratic time complexity ($O(n^2)$) in the number of products [43].

To mitigate this issue, most pipelines employ a filtering-verification based

Approach	Strengths	Limitations
Embedded metadata	Structured, low overhead	Inconsistent adoption
Vision-based	Human-like layout perception	High rendering cost
DOM node classification	Zero-shot, structural reasoning	Complex training data
Markup & layout	High accuracy, lightweight	Domain shift degradation
Multimodal	Robust on visually complex pages	Computationally expensive
LLM-based	Zero-shot, flexible	Token limits, cost, noise sensitivity

Table 2.5: Summary of web information extraction approaches

approach, where a blocking step is used to reduce the number of candidate pairs, followed by a verification step to determine if the candidate pairs are indeed matches. The filtering step usually employs a computationally inexpensive method, with low recall, to quickly eliminate a large number of non-matching pairs, while the verification step uses a more sophisticated method, with higher recall, to accurately identify the matching pairs from the candidate set.

Moreover, as the data we have is unstructured, noisy, and often incomplete, product matching is a challenging task that requires sophisticated techniques to achieve high accuracy.

2.5.1 Blocking and filtering techniques

Blocking involves grouping documents into blocks based on certain criteria, such as shared attributes or similar values, to reduce the number of candidate pairs that need to be compared in the verification step.

Blocking Key Values (BKV)

Blocking Key Values is a traditional blocking technique that uses specific attributes (keys) of an entity to create blocks. For example, in product matching, the blocking key could be the product brand, category, or a combination of attributes. While this technique is straightforward and simple to implement, it can lead to a high number of false negatives if the blocking key is not chosen carefully, or if the data is noisy or contains typos. Another drawback of BKV is that, it requires the key attribute to be present and correctly formatted in the data, which is not always the case in real-world scenarios. [44]

Sorted Neighbour Algorithm (SNA)

The Sorted Neighbour Algorithm is a more sophisticated blocking technique that sorts the records based on a specific attribute and then compares only the records that are within a certain distance of each other in the sorted list. To minimize the effect of noisy data, SNA sorts the records based on a key, and moves a sliding window against it, comparing only the records that fall within the window. [44]

Canopy Clustering

uses two distance thresholds, T_1 and T_2 (where $0 < T_2 < T_1 < 1$): records within T_1 of a randomly selected center are added to its canopy, while those within the tighter T_2 are removed from the candidate pool entirely. This iterative process continues until the pool is exhausted, with records potentially belonging to multiple canopies due to the looser T_1 threshold. The coarse similarity measure used for grouping (commonly TF-IDF distance) is computationally inexpensive, with the effective number of comparisons reduced to approximately $O(fn^2/c)$, where f is the average canopy membership per record and c is the total number of canopies. However, canopy clustering is sensitive to the value selected for T_1. [44]

Locality Sensitive Hashing (LSH)

Locality Sensitive Hashing is a technique that hashes similar items into the same buckets. LSH uses a family of hash functions that are designed to maximize the probability of collision for similar items, while minimizing the

probability of collision for dissimilar items. In most cases, a vector representation (such as TF-IDF, or document embeddings) of the data is used, and the hash functions are designed to preserve the similarity between the vectors. A benefit of LSH is that it can be pre-computed and stored in a hash table, which allows for fast retrieval of candidate pairs. [45]

Pairwise string and text similarity measures

This technique uses various string similarity measures to identify potential candidates for a given record. These can involves **character-based metrics** such as Levenshtein distance, Jaro-Winkler distance, and Hamming distance, which measure the similarity between two strings based on their character-level differences. Alternatively, **token-based metrics** such as Jaccard similarity, TF-IDF cosine similarity, and SoftTF-IDF, which measure the similarity based on the tokens (words) in the strings. [46]

Comparison of blocking and filtering techniques

Technique	Strengths	Limitations
Blocking Key Values	Computationally efficient; can achieve $O(n)$ complexity with high-quality keys	Highly sensitive to noise and typos; a single character error can cause a missed match; requires labelled attributes
Sorted Neighbour Algorithm	More noise-tolerant than BKV; $O(n \log n)$ for sorting, $O(wn)$ for the sliding window	Depends on sorting key quality; requires labelled attributes
Canopy Clustering	Reduces comparisons using coarse similarity; overlapping blocks improve recall; does not require labelled attributes	Sensitive to threshold T_1; complexity is typically $O(n^2)$
Locality Sensitive Hashing	Efficient for high-dimensional vectors; enables fast ANN search; does not require labelled attributes	Approximate by nature; true matches may be missed; complex to tune
Pairwise String Similarity	Flexible; does not require labelled attributes	$O(n^2)$ if used standalone

Table 2.6: Comparison of blocking and filtering techniques

2.5.2 Verification techniques

Once a candidate set of potential matches has been generated through blocking/filtering, the next step is to verify which of these candidates are indeed matches. This step typically involves more computationally expensive techniques, as the number of candidate pairs is significantly reduced compared to the original dataset.

Approximate Nearest Neighbour (ANN) with vector databases

ANN algorithms are designed to efficiently find the nearest neighbors of a given query point in high-dimensional spaces. They essentially trade recall for speed, by allowing for approximate matches rather than exact matches. In the context of product matching, ANN can be used to compare vector representations of candidate product records (such as embeddings, or TF-IDF representation) to determine if they are likely to be matches. There are multiple ANN algorithms available.

Hierarchical Navigable Small World (HNSW) is a state-of-the-art ANN algorithm that constructs a multi-layer graph where each layer represents a different level of granularity in the search space. The algorithm navigates through the graph to find the nearest neighbors efficiently, with a time complexity of $O(\log n)$ for search and $O(n \log n)$ for construction. [47]

Inverted File with Flat Compression (IVF-Flat) is an ANN algorithm that partitions the vector space into a specified number of cluster using the K-means algorithm. At query time, the system identified and searches only the nearest clusters instead of the entire index. This approach significantly reduces the search space, with a time complexity of $O(n \log n)$ for index construction and $O(k \log n)$ for search, where k is the number of clusters searched. [48]

These algorithms can be implemented on PostgreSQL databases using extensions like pgvector [48] and pgvectorscale, which provides efficient indexing and querying capabilities for high-dimensional vectors. [49] However, using dedicated vector databases such as Pinecode, Milvus, or Qdrant can offer faster search performance and better scalability, as they are optimized for ANN search and can handle larger datasets more efficiently. They also support a wider range of ANN algorithms and provide additional features such as distributed indexing, querying, and hybrid search. [50] However for smaller datasets, using a PostgreSQL database with pgvector can be a cost-effective and competive solution [51]

Probabilistic record linkage

Probabilistic record linkage is a technique that uses statistical models to estimate the probability that two records refer to the same entity. This approach typically involves calculating a similarity score for each pair of records based on their attributes, and then using a threshold to determine

if the pair is a match or not. It computes the likelihood of a match based on the agreement and disagreement of attributes, and can incorporate prior knowledge about the data to improve accuracy using Bayesian mechanics. [52]

In essence, it computes three probabilities for whether two records from the dataset are a match:

- m: the probability that a given attribute agrees between two records, given that they are a match.
- u: the probability that a given attribute agrees between two records, given that they are not a match.
- λ: the prior probability that any two records are a match.

For each attribute, the **Bayes Factor** K quantifies the strength of evidence for a match as the ratio of these probabilities:

$$K = \frac{m}{u} \tag{2.1}$$

A Bayes Factor of 10 means an agreement on that attribute is ten times more likely if the records are a match than if they are not. To make weights additive across attributes, the Bayes Factor is log-transformed into a **match weight**:

$$w = \log_2\left(\frac{m}{u}\right) \tag{2.2}$$

A positive weight provides evidence for a match, while a negative weight provides evidence against it. The total match weight $W = \sum_i w_i$ across all attributes is then converted into a final **match probability** via:

$$P(\text{match}) = \frac{\lambda \cdot 2^W}{1 - \lambda + \lambda \cdot 2^W} \tag{2.3}$$

For example, with a uniform prior $\lambda = 0.5$, a total weight of $W = 3$ corresponds to a match probability of approximately 90%, while $W = 7$ yields approximately 99%. [53]

A drawback of using probabilistic record linkage is that it requires attribute-value pairs to be present and correctly formatted in the data, which is not always the case in real-world scenarios. Additionally, it can be computationally expensive to calculate the similarity scores and probabilities for large datasets, especially if the number of attributes is large.

Deep learning and pre-trained language models

Deep learning techniques, particularly those leveraging pre-trained language models (PLMs) such as BERT, have shown significant promise in entity resolution tasks, including product matching. These models can capture complex semantic relationships between product attributes and descriptions, making them particularly effective in handling noisy and unstructured data. By fine-tuning PLMs on labeled datasets of product pairs, the models can learn to generate embeddings that reflect the similarity between products, which can then be used for both blocking and verification steps.

DeepMatcher [54] is a framework which introduced a deep learning-based approach to entity resolution, using customized Recurrent Neural Networks (RNNs) and attention mechanisms to learn representations of product pairs and predict matches. It aggregates attribute values into vector representations before performing pairwise comparisons. While DeepMatcher demonstrated good performance on unstructured data, it can struggle with long sequences of text.

The DITTO framework [55] treats entity resolution and a sequence-to-sequence binary classification task. It serializes the attribute-value pairs of two records into a single string, which is then passed through a transformer model such as BERT or RoBERTa to preict whether the two records match. This technique has shown stronger performance than DeepMatcher, however it requires attribute-value pairs to be present in the data.

RexBERT [56] is a context-specific model based on BERT which has been fine-tuned on an e-commerce product dataset. While general purpose pre-trained language models can capture semantic relationships, they may not be optimized for the specific language and structure of product data. RexBERT is designed to better capture the nuances of product descriptions and attributes, which can lead to improved performance in product matching tasks.

Multi-modal approaches

Multi-modal approaches involve using both textual and visual information to perform product matching. For example, a product may have a textual description as well as an image, and both of these can be used to determine if two products are a match. By combining features from both modalities, multi-modal approaches can often achieve higher accuracy than methods that rely solely on text or images. For instance, a product may have a similar de-

scription to another product, but the images may reveal that they are different products (e.g., different colors or designs). Conversely, two products may have different descriptions but similar images, which could indicate that they are the same product with different attribute values (e.g., different sizes or materials). While this technique is significantly more complex and resource intensive than text-only approaches, its effectiveness in e-commerce datasets may not be as expected, as different product variations can have very similar images, and product descriptions can often contain more discriminative information than images alone. [57]

Comparison of verification techniques

Technique	**Strengths**	**Limitations**	**Requires Attr. Data?**
HNSW	Graph-based; $O(\log n)$ search; high recall, low latency	Memory-intensive; slow index build	No
IVFFlat	Faster builds and lower memory than HNSW; suits frequent re-indexing	Lower recall; depends on cluster count	No
Probabilistic Record Linkage	Weighs multiple fields; interpretable; robust on messy data	Requires tuning of m, u, and thresholds	Yes
DeepMatcher	Semantic similarity via RNNs and attention; strong on dirty data	Slow training; OOM errors on large datasets	No
DITTO	Strong performance; label-efficient	512-token limit; high GPU cost	Yes
RexBERT	E-commerce optimised; 8,192-token context; efficient vs. larger models	Domain-specific; needs large retail corpus	No
Multi-modal	Fuses text and image; effective in visual domains	High compute cost; weak image signal in some domains	No

Table 2.7: Comparison of verification techniques for product matching

2.6 Related work

2.6.1 Consumer-facing public systems

Several publicly accessible systems exist which aggregate product information from multiple online retailer, and make it available to consumers. They offer different features and the underlying techniques used to extract and process the data vary widely.

Some of the most popular systems include:

- **CamelCamelCamel**[1] tracks over 18 million products on Amazon, generating over 72 million price records per day. While it primarily uses the Amazon Product Advertising API to collect product information, it also employs distributed headless web scraping to extract information when API rate-limits are reached, and also to extract unexposed metrics such as Buy Box variations. A major drawback of CamelCamelCamel is that it is limited to the Amazon ecosystem. [58]
- **Keepa**[2] monitors over 5.6 billion products over 11 global marketplaces. It provides detailed charts including sales rank, Buy Box statistics, and inventory levels, updated as frequently as every 15 minutes. It provides deeper metrics such as sales rank history, inventory leve, and Buy Box statistics. While its free-tier offers basic features, the more advanced features require a premium subscription. [59]
- **PayPal Honey**[3] operates across thousands of retailers. Its droplist feature allows users to track price changes for specific products, and it also provides a browser extension that automatically applies coupon codes at checkout. Honey collects product information through a combination of web scraping and partnerships with retailers, and it also uses crowdsourced data from its user base to enhance its product database. [60]
- **37left.lk**[4] is a Sri Lankan price comparison website that allows users to compare prices of products across multiple online retailers and check

[1] `https://camelcamelcamel.com/`
[2] `https://keepa.com/`
[3] `https://www.joinhoney.com/`
[4] `https://37left.lk/`

historical price records. It collects product information by continuously crawling the websites of participating retailers. It also supports features such as price alerts, allowing users to receive notifications when the price of a tracked product drops below a specified threshold. Currently, 12 Sri Lankan retailers are supported. [61]

2.6.2 Self-hosted and open-source systems

Several self-hosted and open-source systems exist that allow users to track product prices and availability across multiple online retailers. These systems often provide more flexibility and control over the data collection and processing methods, but may require more technical expertise to set up and maintain. A key benefit that self-hosted systems offer is that they allow users to keep control of their data and privacy.

- **PriceGhost** allows price tracking from virtually any e-commerce website. Its primary highlight, is its Price Voting Modal, which runs four independent extraction methods in parallel to extract product information from a given product page. [62]
  - JSON-LD: Reads structured data embedded in the product page's HTML, which is often used by e-commerce websites to provide product information in a machine-readable format.
  - Site-specific scrapers: Using custom rule-based scrapers for major retailers such as Amazon, Walmart, and Best Buy.
  - Generic CSS: Using intelligent CSS selectors to extract product information from the page's HTML.
  - AI Analysis: Using LLMs to analyze the product page and extract relevant information based on the page's content and structure.

  The results from these four methods, showing each candidate with the method used and the confidence score, are presented to the user for verification and selection.

- **PriceBuddy**, while it also supports price tracking for major retailers out of the box, it also allows users to create custom scrapers for any e-commerce website by providing CSS selectors, regular expressions, or JSON paths. It also offers a multi-store comparison feature, which allows users to add multiple URLs for a single product entry and compare

prices across different retailers. A distinct feature of PriceBuddy is its unit pricing feature, which allows users to compare prices based on the quantity or size of the product, making it easier to identify the best value per unit. It also provides support for in-stock tracking, including opt-in back-in-stock alerts. [63]

2.6.3 Enterprise competitive intelligence systems

Several enterprise-level competitive intelligence systems exist that provide businesses with insights into their competitors' pricing strategies, product offerings, and market trends. These systems often offer advanced features such as automated data collection, analytics dashboards, and integration with other business intelligence tools. They are typically used by larger organizations with dedicated teams for market research and competitive analysis

- **Omnia Retail**[5] provide intelligence platforms combining dynamic repricing with agentic AI. Omnia Agent analyzes pricing data and surfaces relevant insights without requiring manually curated dashboards, offering recommendations. These systems help organizations spot pricing shifts in real-time and execute pricing strategies across multiple markets. [64]
- **Price2Spy**[6] specializes in Minimum Advertised Price (MAP) monitoring and enforcement. It helps brands maintain pricing integrity across global reseller networks through automated evidence collection and real-time monitoring of reseller compliance. [64]
- **Competera**[7] is an AI-native platform combining elasticity-based pricing with predictive algorithms. It suggests optimal pricing adjustments based on customer behavior and market conditions, reducing operational costs by up to 30% while improving pricing accuracy and customer lifetime value. [65] The platform excels in product matching and relationship management through several capabilities:
  - *Product Relationship Management*: Establishes linear or hierarchical dependencies between products, allowing synchronized pricing adjustments across related items.

[5] `https://www.omniaretail.com`
[6] `https://www.price2spy.com`
[7] `https://www.competera.net`

– *Dynamic Product Assignment*: Automatically distributes products to pricing campaigns based on specific sales parameters and business constraints.

2.6.4 Summary of Related Systems

System	Type	Key Features	Primary Techniques Used
CamelCamelCamel	Public (Free)	Amazon-exclusive price tracking, historical price charts, and email price-drop alerts	Web scraping and official Amazon Product Advertising API
Keepa	Public (Freemium)	Detailed Amazon price/stock history, sales rank tracking, Buy Box statistics, and inventory monitoring	Continuous large-scale scraping of billions of products and API access for premium data
PayPal Honey	Public (Free)	Automated coupon finder (primary), multi-retailer "Droplist" for tracking, and Amazon seller price comparisons	Browser extension tracking, affiliate-driven data model, and alleged "Secret Tab" redirection for attribution
37left.lk	Public (Free)	Multi-store price comparison specifically for Sri Lanka, 24/7 scanning, and WhatsApp/email alerts	Automated retailer scanning, lightning-fast internal search engine, and synonym matching
PriceGhost	Open-Source (Self-Hosted)	Universal price/stock tracking, interactive history charts, multi-user support, and back-in-stock alerts	"Price Voting" system using four parallel methods: JSON-LD, site-specific scrapers, generic CSS, and AI fallback
PriceBuddy	Open-Source (Self-Hosted)	Support for custom store addition (CSS/Regex), side-by-side multi-store comparison, and unit price calculation	Scrapers based on custom selectors/JSONPath, headless browser rendering (SeleniumBase), and AI self-healing
Omnia Retail	Enterprise	Multi-channel competitor monitoring, dynamic repricing, MAP enforcement, and Buy Box analytics	Real-time monitoring, direct API connections to comparison engines, and agentic AI for shift detection
Price2Spy	Enterprise	Specialized MAP monitoring with evidence collection, global marketplace tracking, and detailed history	4-tier matching (Manual, Automatch, Quick, ML Hybrid), hourly refresh rates, and manual QA validation
Competera	Enterprise	AI predictive pricing, high-accuracy exact and similar product matching, and demand elasticity modeling	Deep Learning for entity resolution, real-time client-specific crawling infrastructure, and AI pricing assistants

Table 2.8: Comparison of price-tracking and product-matching systems.

Bibliography

[1] P. Reyes, "4 types of website structures (detailed explanation)," Dec 2025.

[2] Wix Staff, "Parts of a website." `https://www.wix.com/blog/parts-of-a-website`, 2026. Accessed: May 20, 2026.

[3] Ricca and Tonella, "Web site analysis: Structure and evolution," in *Proceedings 2000 International Conference on Software Maintenance*, pp. 76–86, IEEE, 2000.

[4] GeeksforGeeks, "Javascript html dom." `https://www.geeksforgeeks.org/html/javascript-html-dom/`, 2026. Accessed: May 20, 2026.

[5] Yotpo, "The e-commerce marketing funnel: A complete guide." `https://www.yotpo.com/blog/ecommerce-marketing-funnel/`, 2026. Accessed: May 20, 2026.

[6] M. Lanthaler and C. Gütl, "On using json-ld to create evolvable restful services," in *Proceedings of the third international workshop on RESTful design*, pp. 25–32, 2012.

[7] M. Kumar, R. Bhatia, and D. Rattan, "A survey of web crawlers for information retrieval," *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, vol. 7, no. 6, p. e1218, 2017.

[8] M. A. Kausar, V. Dhaka, and S. K. Singh, "Web crawler: a review," *International Journal of Computer Applications*, vol. 63, no. 2, pp. 31–36, 2013.

[9] A. Goel, J. Zhu, R. Netravali, and H. V. Madhyastha, "Sprinter: Speeding up High-Fidelity crawling of the modern web," in *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, (Santa Clara, CA), pp. 893–906, USENIX Association, Apr. 2024.

[10] A. Mesbah, A. Van Deursen, and S. Lenselink, "Crawling ajax-based web applications through dynamic analysis of user interface state changes," *ACM Transactions on the Web (TWEB)*, vol. 6, no. 1, pp. 1–30, 2012.

[11] Z. Chang, "A survey of modern crawler methods," in *Proceedings of the 6th international conference on control engineering and artificial intelligence*, pp. 21–28, 2022.

[12] M. A. Khder, "Web scraping or web crawling: State of art, techniques, approaches and application.," *International Journal of Advances in Soft Computing & Its Applications*, vol. 13, no. 3, 2021.

[13] K. Kothari, "Web crawler development: How to build a custom crawler," Mar 2026.

[14] Cloudflare, "Cloudflare bot management," 2026.

[15] Google, "Things to know about google's web crawling — google crawling infrastructure — crawling infrastructure — google for developers," 2026.

[16] J. Pomikálek, "Removing boilerplate and duplicate content from web corpora," *Disertacnı práce, Masarykova univerzita, Fakulta informatiky*, 2011.

[17] L. Yi, B. Liu, and X. Li, "Eliminating noisy information in web pages," in *ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD-2003)*, pp. 24–27.

[18] D. Gibson, K. Punera, and A. Tomkins, "The volume and evolution of web page templates," in *Special interest tracks and posters of the 14th international conference on World Wide Web*, pp. 830–839, 2005.

[19] T. Vogels, O.-E. Ganea, and C. Eickhoff, "Web2text: Deep structured boilerplate removal," in *European Conference on Information Retrieval*, pp. 167–179, Springer, 2018.

[20] C. Kohlschütter, P. Fankhauser, and W. Nejdl, "Boilerplate detection using shallow text features," in *Proceedings of the third ACM international conference on Web search and data mining*, pp. 441–450, 2010.

[21] R. Schäfer, "Accurate and efficient general-purpose boilerplate detection for crawled web corpora," *Language Resources and Evaluation*, vol. 51, no. 3, pp. 873–889, 2017.

[22] J. Leonhardt, A. Anand, and M. Khosla, "Boilerplate removal using a neural sequence labeling model," in *Companion Proceedings of the Web Conference 2020*, pp. 226–229, 2020.

[23] B. Ajose-Ismail and Q. Osanyin, "A systematic review on web page classification," *International Journal of Computer Trends and Technology*, vol. 68, no. 4, pp. 81–86, 2020.

[24] Oct 2025.

[25] F. De Fausti, F. Pugliese, and D. Zardetto, "Towards automated website classification by deep learning," *arXiv preprint arXiv:1910.09991*, 2019.

[26] W. Petprasit and S. Jaiyen, "E-commerce web page classification based on automatic content extraction," in *2015 12th International joint conference on computer science and software engineering (JCSSE)*, pp. 74–77, IEEE, 2015.

[27] A. Mahara, "Cybersecurity data extraction from common crawl," *arXiv preprint arXiv:2602.22218*, 2025.

[28] H. Yu, J. Han, and K.-C. Chang, "Pebl: Web page classification without negative examples," *IEEE Transactions on Knowledge and Data Engineering*, vol. 16, no. 1, pp. 70–81, 2004.

[29] D. Shen, Z. Chen, Q. Yang, H.-J. Zeng, B. Zhang, Y. Lu, and W.-Y. Ma, "Web-page classification through summarization," in *Proceedings of the 27th annual international ACM SIGIR conference on Research and development in information retrieval*, pp. 242–249, 2004.

[30] J. Li, Y. Xu, L. Cui, and F. Wei, "Markuplm: Pre-training of text and markup language for visually rich document understanding," in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 6078–6087, 2022.

[31] J. Ronallo, "Html5 microdata and schema. org," *Code4Lib Journal*, no. 16, 2012.

[32] B. Adida, M. Birbeck, S. McCarron, and S. Pemberton, "Rdfa 1.1 primer - third edition," w3c working group note, World Wide Web Consortium (W3C), March 2015. Retrieved May 22, 2026.

[33] M. Sporny, G. Kellogg, and M. Lanthaler, "Json-ld 1.1: A json-based serialization for linked data," w3c recommendation, World Wide Web Consortium (W3C), July 2020. Retrieved May 22, 2026.

[34] "Schema.org: Shared vocabularies for structured data." https://schema.org/, 2026. Retrieved May 22, 2026.

[35] D. Tierney, "Analysing schema implementation across 100 top ecommerce websites." https://salt.agency/blog/structured-data-implementation-across-top-100-ecommerce-sites/, June 2023. Retrieved May 23, 2026.

[36] D. Cai, S. Yu, J.-R. Wen, and W.-Y. Ma, "Vips: a vision-based page segmentation algorithm," 2003.

[37] C. Lockard, P. Shiralkar, X. L. Dong, and H. Hajishirzi, "Zeroshotceres: Zero-shot relation extraction from semi-structured webpages," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pp. 8105–8117, 2020.

[38] R. Potta, M. Asthana, S. Yadav, N. Goyal, S. Patnaik, and P. Jain, "Attrisage: Product attribute value extraction using graph neural networks," in *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: Student Research Workshop*, pp. 89–94, 2024.

[39] "Common crawl: Open repository of web crawl data." https://commoncrawl.org/, 2026. Retrieved May 23, 2026.

[40] Q. Wang, Y. Fang, A. Ravula, F. Feng, X. Quan, and D. Liu, "Webformer: The web-page transformer for structure information extraction," in *Proceedings of the ACM Web Conference 2022*, pp. 3124–3133, 2022.

[41] Y. Huang, T. Lv, L. Cui, Y. Lu, and F. Wei, "Layoutlmv3: Pre-training for document ai with unified text and image masking," in *Proceedings of the 30th ACM international conference on multimedia*, pp. 4083–4091, 2022.

[42] M. Vinciguerra, "Llm web scraping: How ai models replace scrapers." https://scrapegraphai.com/blog/llm-web-scraping/, April 2026. Retrieved May 23, 2026.

[43] J. Shao and Q. Wang, "Active blocking scheme learning for entity resolution," in *Pacific-asia conference on knowledge discovery and data mining*, pp. 350–362, Springer, 2018.

[44] M. H. Moslemi, H. Balamurugan, and M. Milani, "Evaluating blocking biases in entity matching," in *2024 IEEE International Conference on Big Data (BigData)*, pp. 64–73, IEEE, 2024.

[45] A. Brinkmann, R. Shraga, and C. Bizer, "Sc-block: Supervised contrastive blocking within entity resolution pipelines," in *European Semantic Web Conference*, pp. 121–142, Springer, 2024.

[46] Y. Sun, L. Ma, and S. Wang, "A comparative evaluation of string similarity metrics for ontology alignment," *Journal of Information &Computational Science*, vol. 12, no. 3, pp. 957–964, 2015.

[47] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE transactions on pattern analysis and machine intelligence*, vol. 42, no. 4, pp. 824–836, 2018.

[48] Instaclustr, "pgvector performance: Benchmark results and 5 ways to boost performance." `https://www.instaclustr.com/education/vector-database/pgvector-performance-benchmark-results-and-5-ways-to-boost-performance/`, 2025. Accessed: 2026-06-02.

[49] Firecrawl, "The best vector databases for 2026." `https://www.firecrawl.dev/blog/best-vector-databases`, 2026. Accessed: 2026-06-02.

[50] Encore.dev, "pgvector vs. qdrant: Choosing the right vector database," 2026. Accessed: 2026-06-02.

[51] M. I. Zaman, "pgvector vs elasticsearch vs qdrant vs pinecone vs weaviate: A 14-case benchmark." `https://huggingface.co/blog/ImranzamanML/pgvector-vs-elasticsearch-vs-qdrant-vs-pinecone-vs`, April 2026. Accessed: 2026-06-02.

[52] Prospeo, "What is probabilistic matching?." `https://prospeo.io/s/probabilistic-matching`, 2026. Accessed: 2026-06-02.

[53] W. E. Winkler and Y. Thibaudeau, *An application of the Fellegi-Sunter model of record linkage to the 1990 US decennial census*. US Bureau of the Census Washington, DC, 1991.

[54] T. Xie, K. Dai, K. Wang, R. Li, and L. Zhao, "Deepmatcher: a deep transformer-based network for robust and accurate local feature matching," *Expert Systems with Applications*, vol. 237, p. 121361, 2024.

[55] Y. Li, J. Li, Y. Suhara, A. Doan, and W.-C. Tan, "Deep entity matching with pre-trained language models," *arXiv preprint arXiv:2004.00584*, 2020.

[56] R. Bajaj and A. Garg, "Rexbert: Context specialized bidirectional encoders for e-commerce," *arXiv preprint arXiv:2602.04605*, 2026.

[57] M. Wilke and E. Rahm, "Towards multi-modal entity resolution for product matching.," in *GvDB*, 2021.

[58] System Design Handbook, "Design a price tracking service like camelcamelcamel." `https://www.systemdesignhandbook.com/guides/design-camelcamelcamel/`, 2026. Accessed: 2026-06-22.

[59] HARPA AI, "The best amazon price trackers and price drop alerts (2026)." `https://harpa.ai/blog/best-amazon-price-trackers-and-drop-alerts`, 2026. Accessed: 2026-06-22.

[60] PayPal Editorial Staff, "The complete guide to using paypal honey." `https://www.paypal.com/us/money-hub/article/guide-to-using-paypal-honey`, Dec. 2025. Accessed: 2026-06-22.

[61] 37Left.lk, "Frequently asked questions." `https://37left.lk/faq`, 2026. Accessed: 2026-06-22.

[62] clucraft, "Priceghost: A self-hosted price tracking application." `https://github.com/clucraft/PriceGhost`, 2026. Accessed: 2026-06-22.

[63] jez500, "Pricebuddy: An automated price tracking utility." `https://github.com/jez500/pricebuddy`, 2026. Accessed: 2026-06-22.

[64] ScrapeWise, "Top competitive price monitoring tools for 2026." `https://scrapewise.ai/blogs/competitive-price-monitoring-tools-2026`, 2026. Accessed: 2026-06-22.

[65] Competera, "Ai-powered product matching solution." `https://competera.ai/solutions/by-need/product-matching`, 2026. Accessed: 2026-06-23.